Question 1: Record all commands, pipeline configurations, screenshots, commit history, and observations for each experiment.

Executed on Ubuntu 24.04 LTS (WSL2) by Meenakshi Rakesh. Repository: https://github.com/meenakshi16rp/Devops-lab-1-

1. Environment and service verification



Figure: Ubuntu terminal: environment and Jenkins service verification (user meenakshi, Ubuntu 24.04.3, Git 2.43.0, Jenkins 2.580.2 active)

2. Commit history (github.com/meenakshi16rp/Devops-lab-1-)

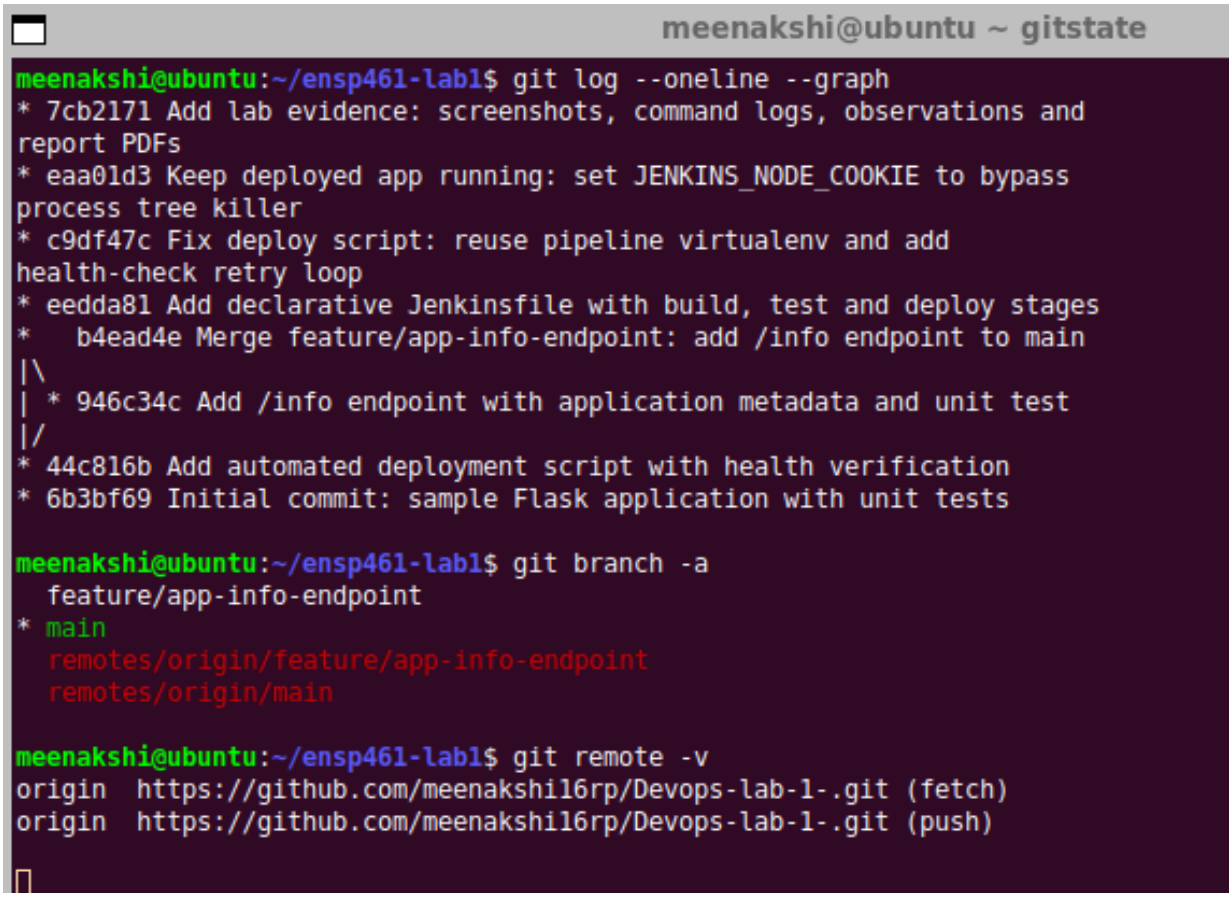


Figure: Ubuntu terminal: resulting commit graph, branches and remotes

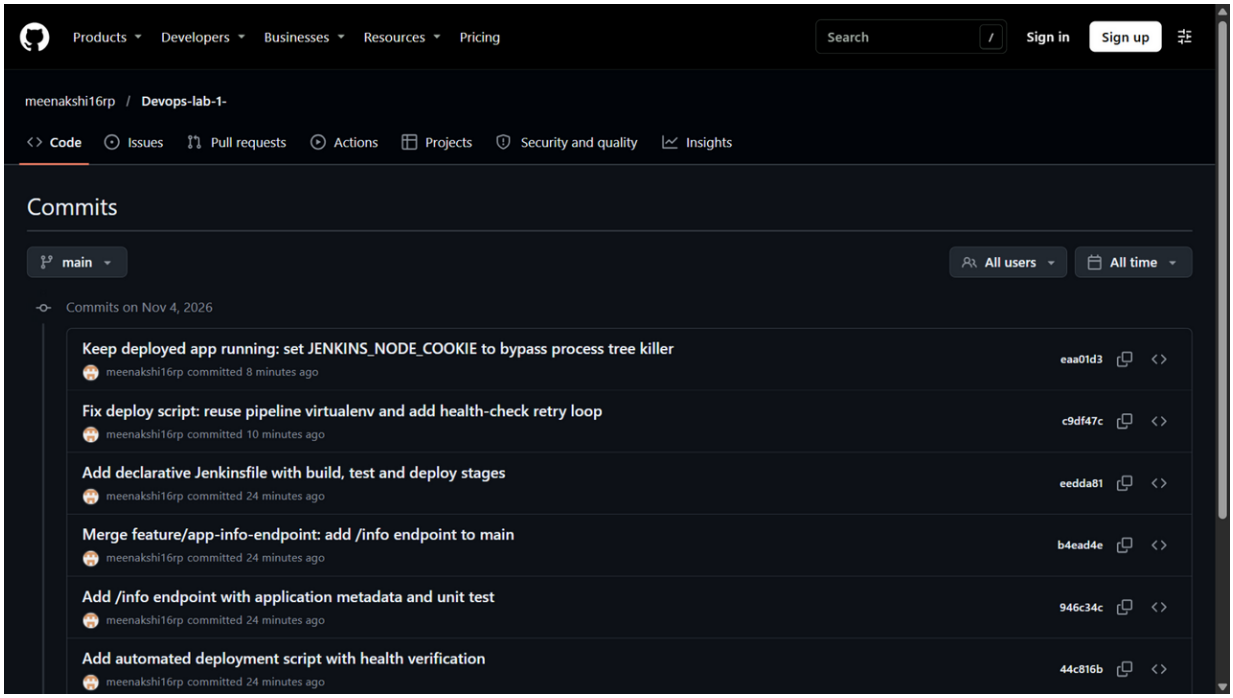


Figure: Commit history on GitHub - meaningful messages, merge commit visible

3. Git installation and configuration (Q4)

installation and global configuration

```
sudo apt update && sudo apt install -y git
git config --global user.name "Meenakshi"
git config --global user.email "meenakshi16rp@gmail.com"
git config --global init.defaultBranch main
```

```
git config --global credential.helper store
git config --global --list
```

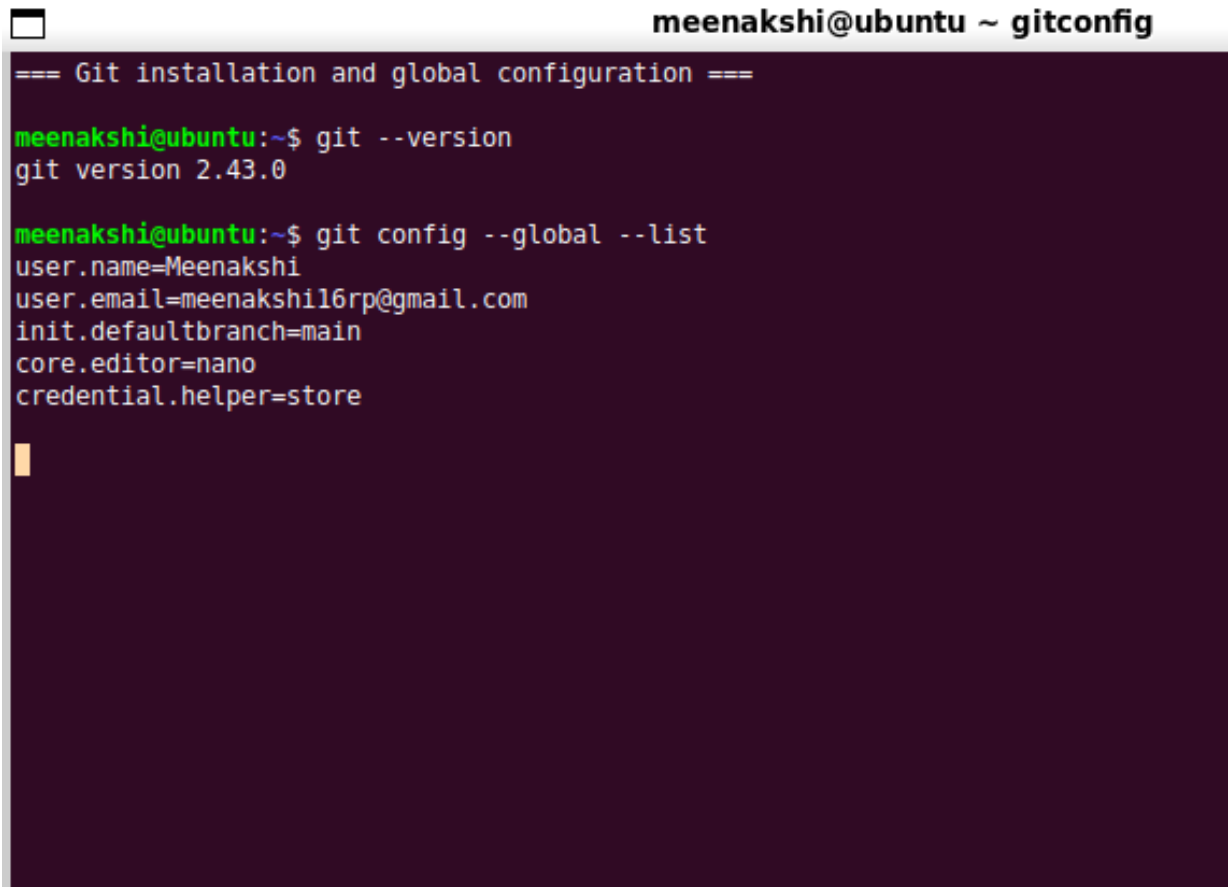


Figure: Terminal verification: user meenakshi, Ubuntu 24.04.3, git 2.43.0 and the global configuration

4. Jenkins installation and first-time setup (Q4)

Jenkins LTS installation (systemd service on port 8080)

```
sudo apt install -y openjdk-21-jdk
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key -o \
      /usr/share/keyrings/jenkins-keyring.asc
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" \
      | sudo tee /etc/apt/sources.list.d/jenkins.list
sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl is-active jenkins
active
jenkins --version
2.580.2
```

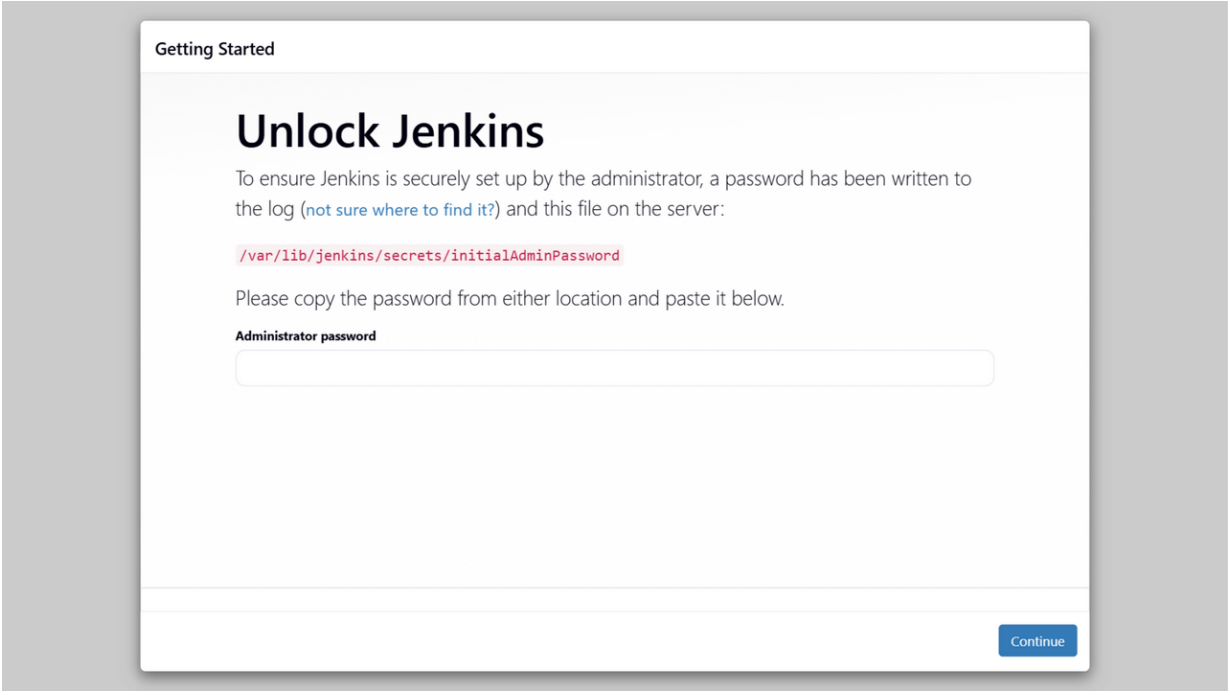


Figure: Unlock Jenkins with the initial admin password

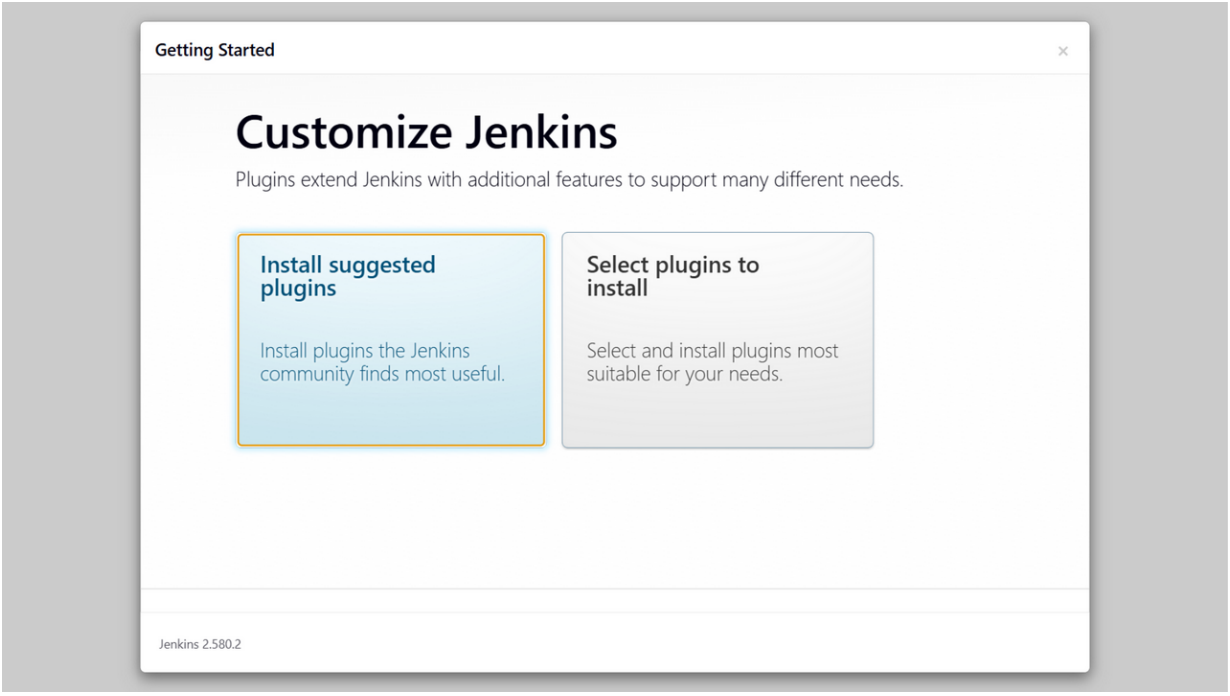


Figure: Customize Jenkins - install suggested plugins

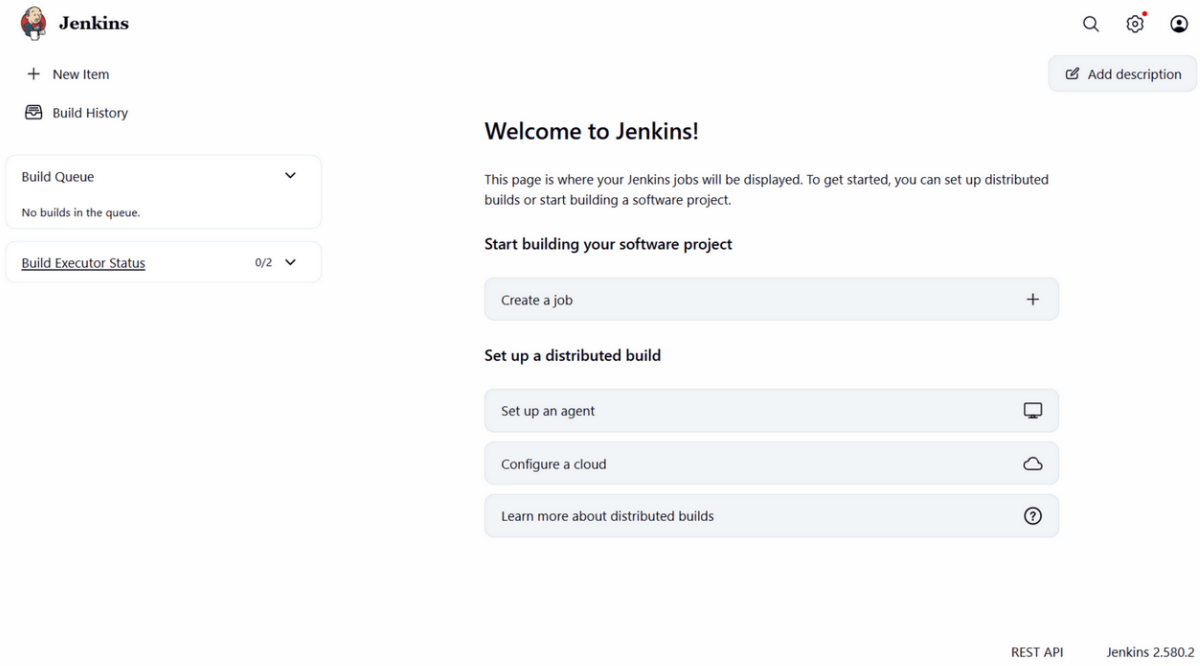


Figure: Jenkins dashboard ready (admin user: meenakshi)

5. Version control workflow (Q5)

init, stage, commit, branch, merge, push

```
cd ~/ensp461-lab1
git init -b main
git add README.md .gitignore app/app.py app/requirements.txt app/tests/test_app.py
git commit -m "Initial commit: sample Flask application with unit tests"
git add deploy/deploy.sh
git commit -m "Add automated deployment script with health verification"
git checkout -b feature/app-info-endpoint
# ... implemented /info endpoint + unit test on the branch ...
git add app/ && git commit -m "Add /info endpoint with application metadata and unit test"
git checkout main
git merge --no-ff feature/app-info-endpoint -m "Merge feature/app-info-endpoint: add /info \
      endpoint to main"
git add Jenkinsfile
git commit -m "Add declarative Jenkinsfile with build, test and deploy stages"
git remote add origin https://github.com/meenakshi16rp/Devops-lab-1-.git
git push -u origin main
git push -u origin feature/app-info-endpoint
```

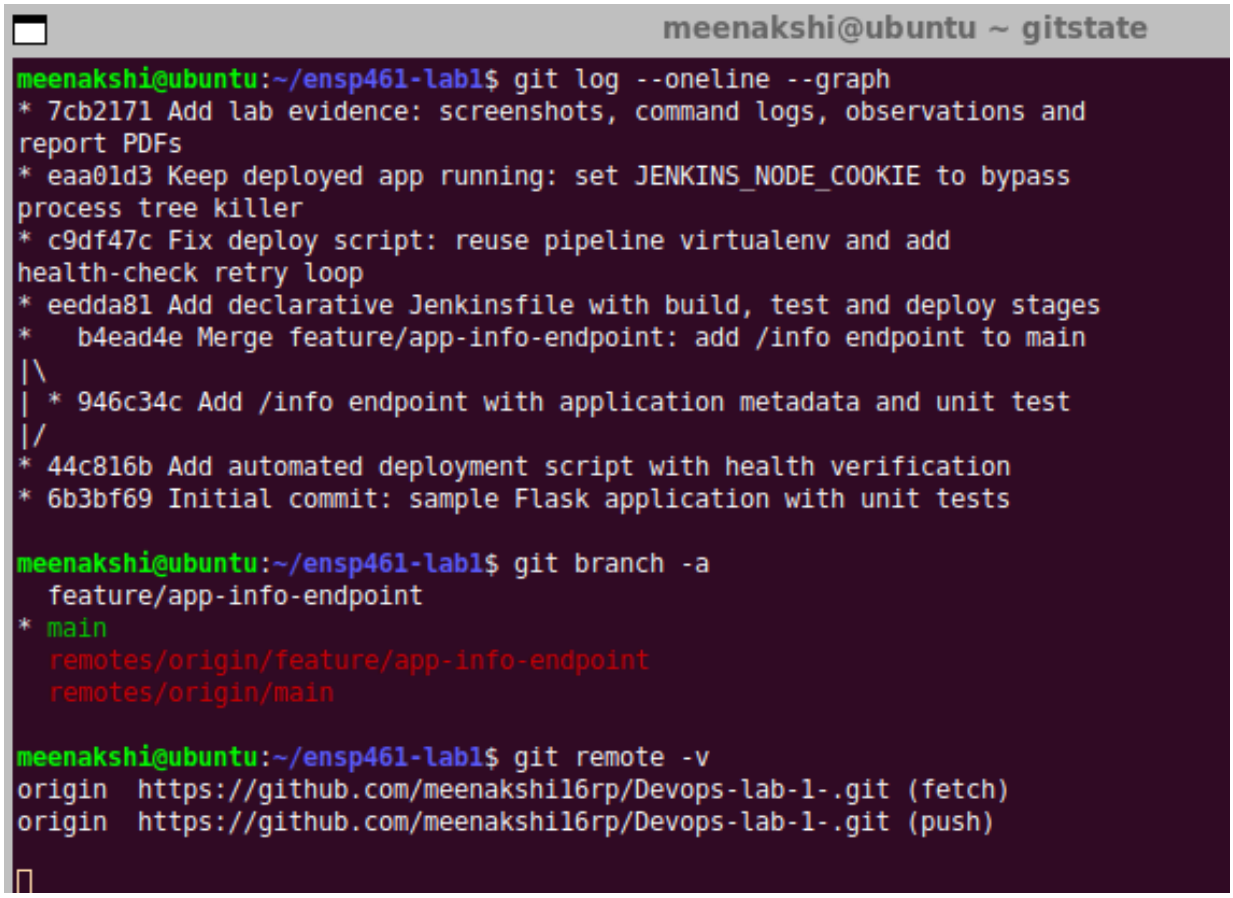


Figure: Ubuntu terminal: resulting commit graph, branches and remotes

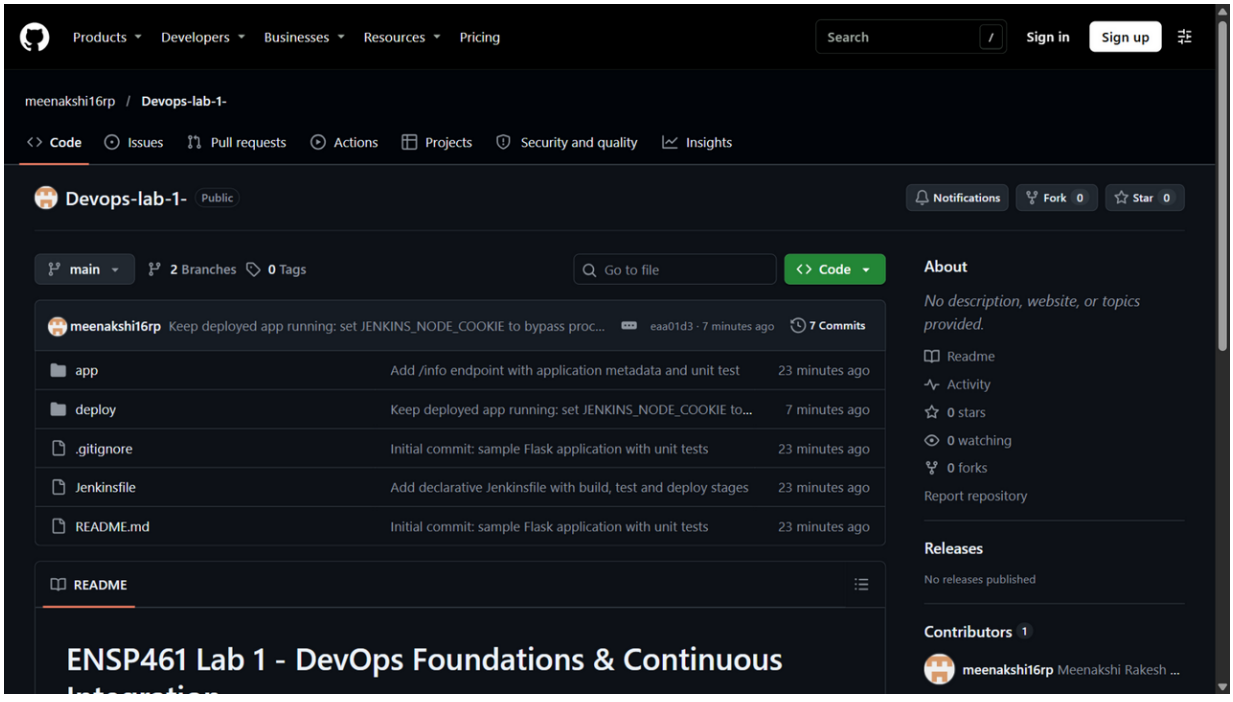


Figure: Repository on GitHub: 8 commits, 2 branches, Jenkinsfile and README

6. Pipeline job and Jenkinsfile execution (Q6/Q7)

job configuration summary

```
Jenkins job:        ensp461-lab1-pipeline (Pipeline)
Definition:         Pipeline script from SCM
SCM:                Git
Repository URL:     https://github.com/meenakshi16rp/Devops-lab-1-.git
```

```
Credentials:        github-creds (meenakshi16rp /****** GitHub PAT)
Branch:             */main
Script path:        Jenkinsfile
Build trigger:      Poll SCM -> H/5 * * * *
Post-build:         post { success / failure / always { cleanWs() } }
```

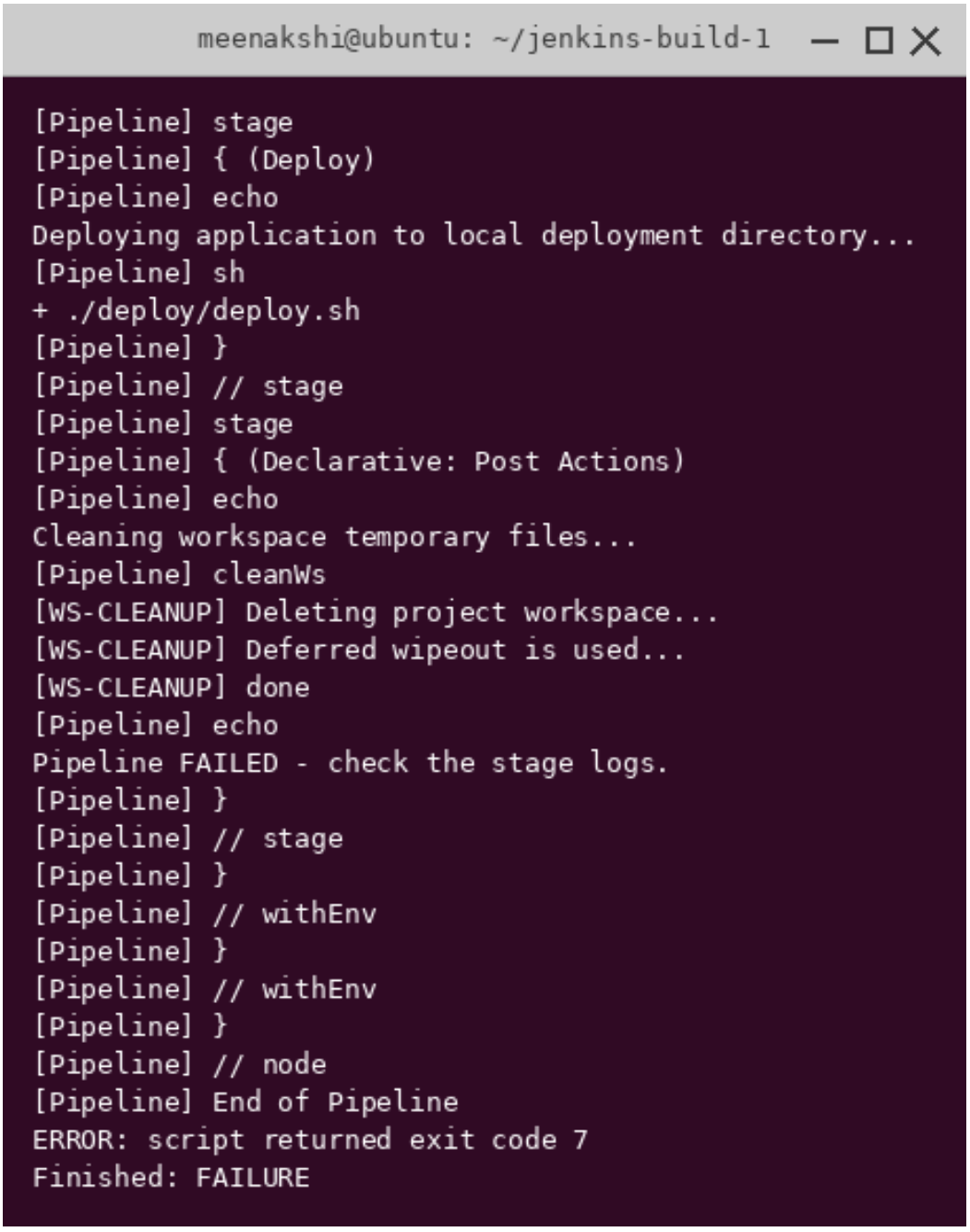


Figure: Build #1 console output (tail) - FAILURE: deploy stage defect

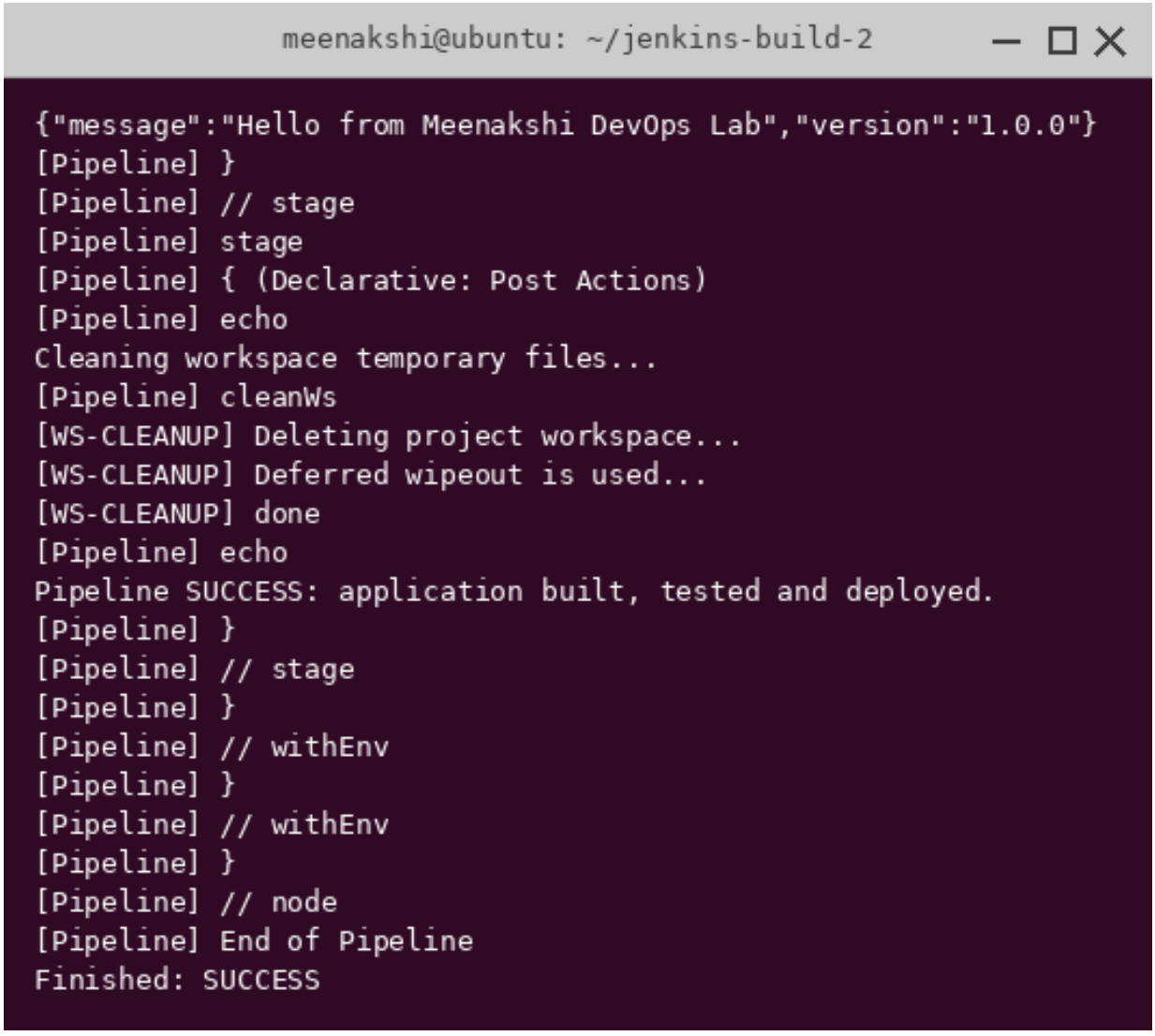


Figure: Build #2 console output (tail) - SUCCESS after the fix

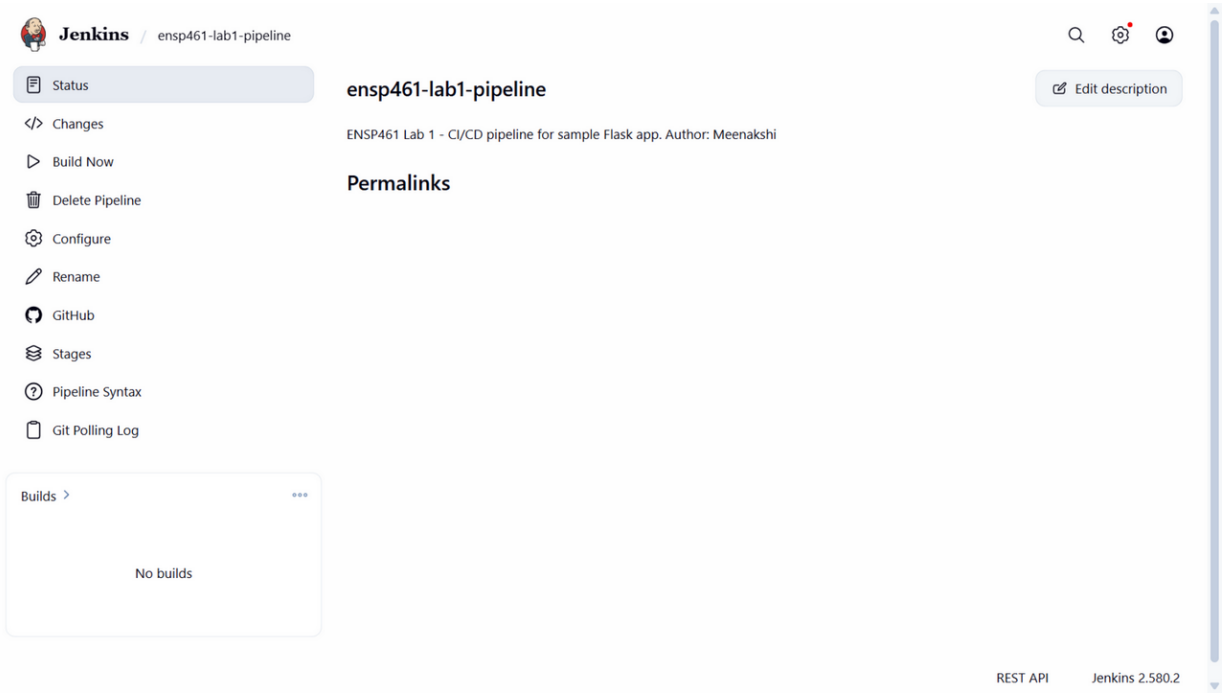


Figure: Pipeline job page (GitHub project link, stages, polling log)

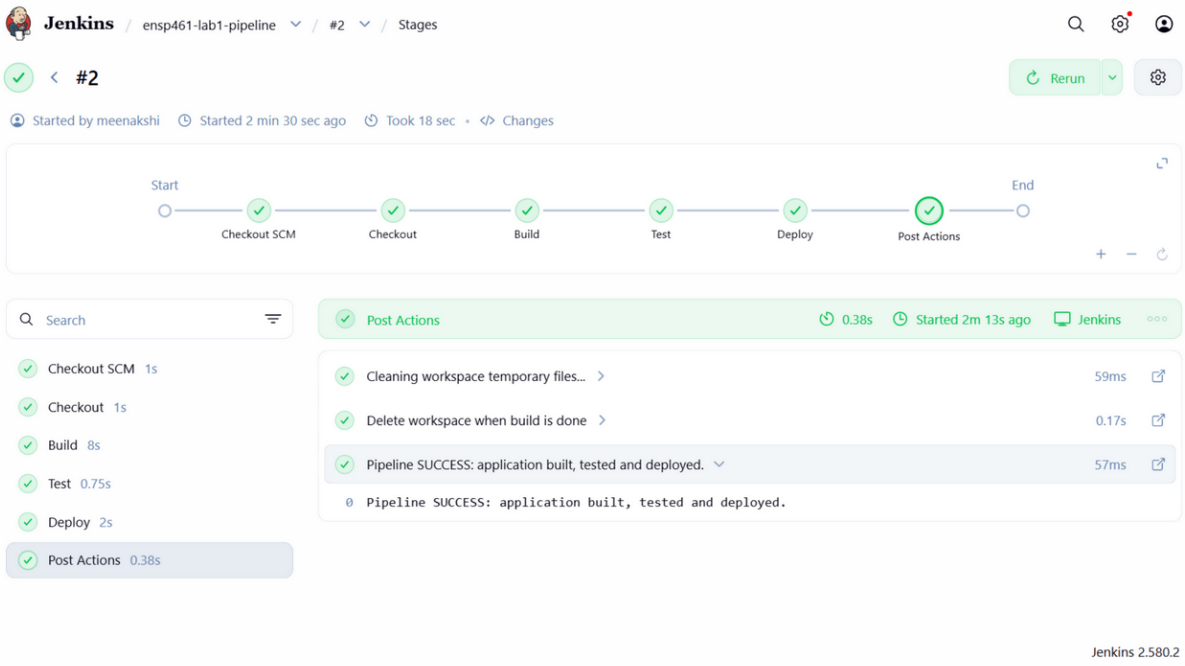


Figure: Build #2 stage graph: all stages green

6. Automated deployment (Q3)

trigger and deployment steps

```
Build trigger: Poll SCM - H/5 * * * *  (every 5 minutes)
deploy step:   ./deploy/deploy.sh  (reuses the Build stage virtualenv,
               restarts the app, verifies http://localhost:5000/health)
```

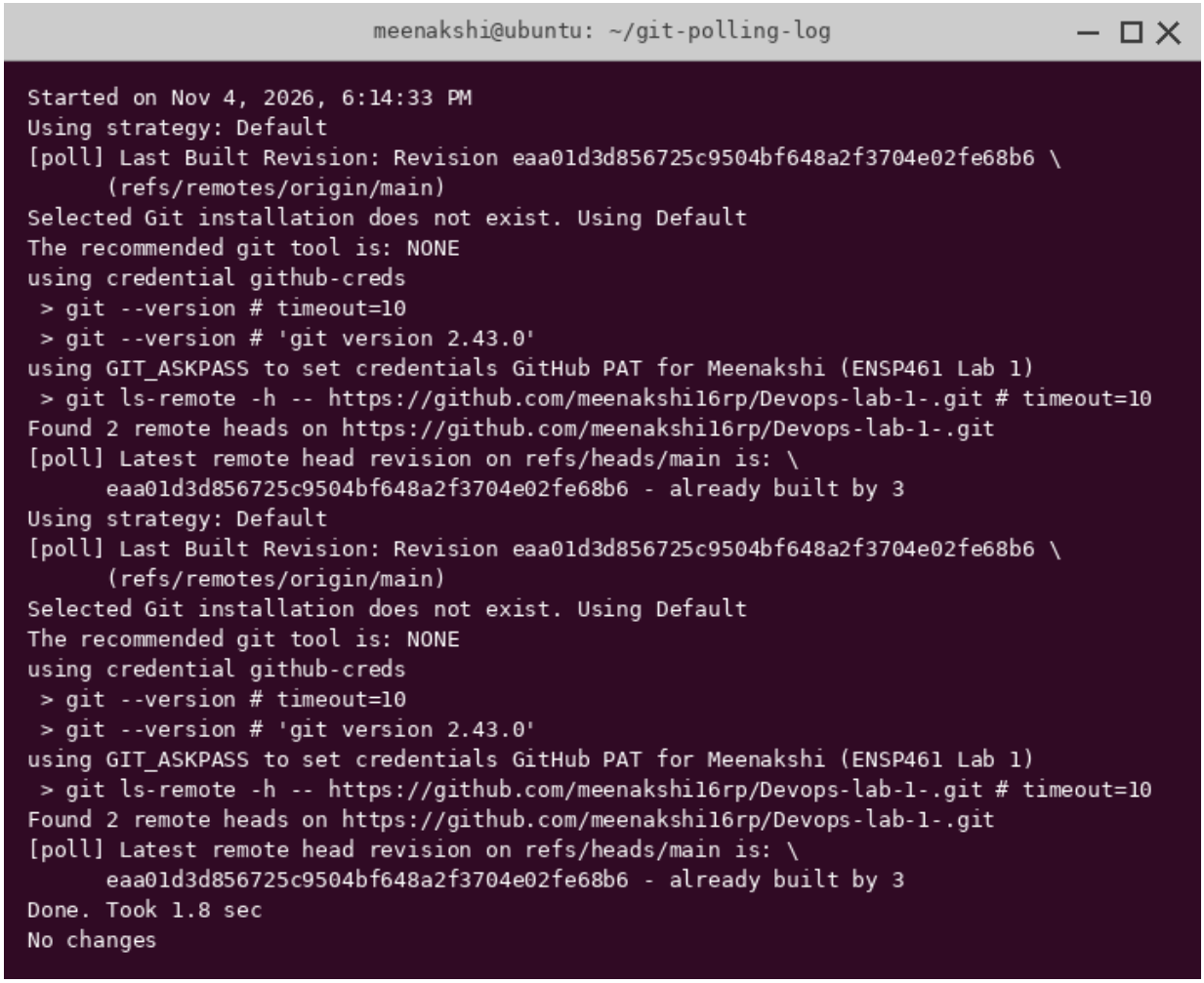


Figure: Git polling log (captured output) - poll detects commit eaa01d3 -> build #3

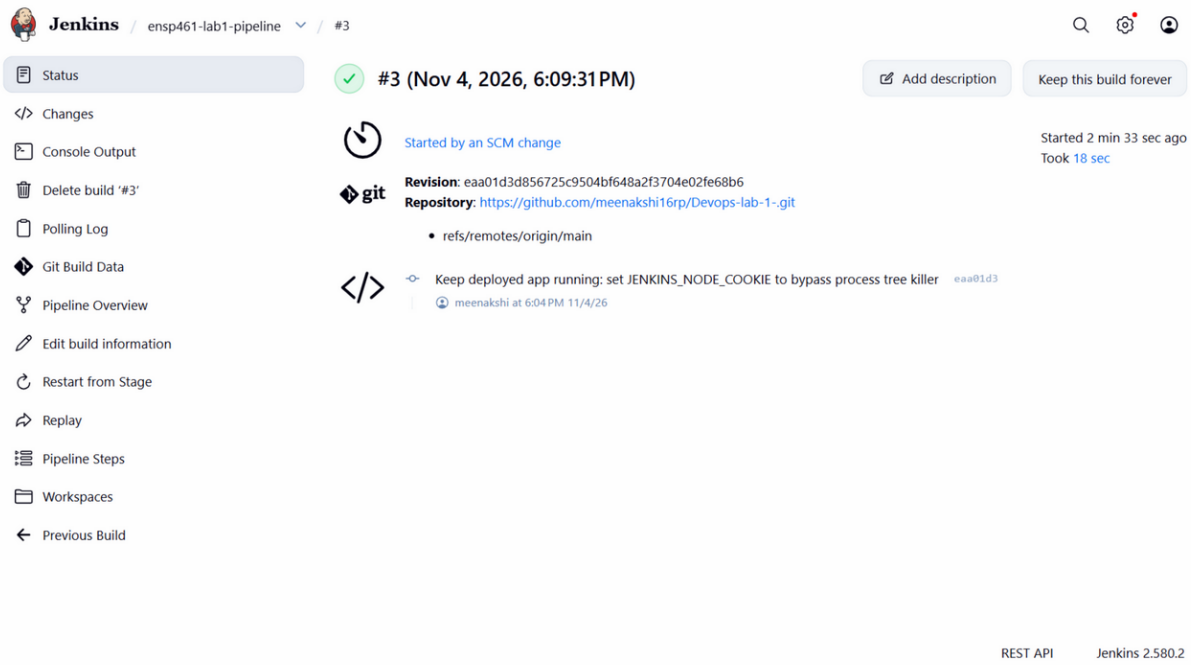


Figure: Build #3 - cause: 'Started by an SCM change' (automatic trigger)

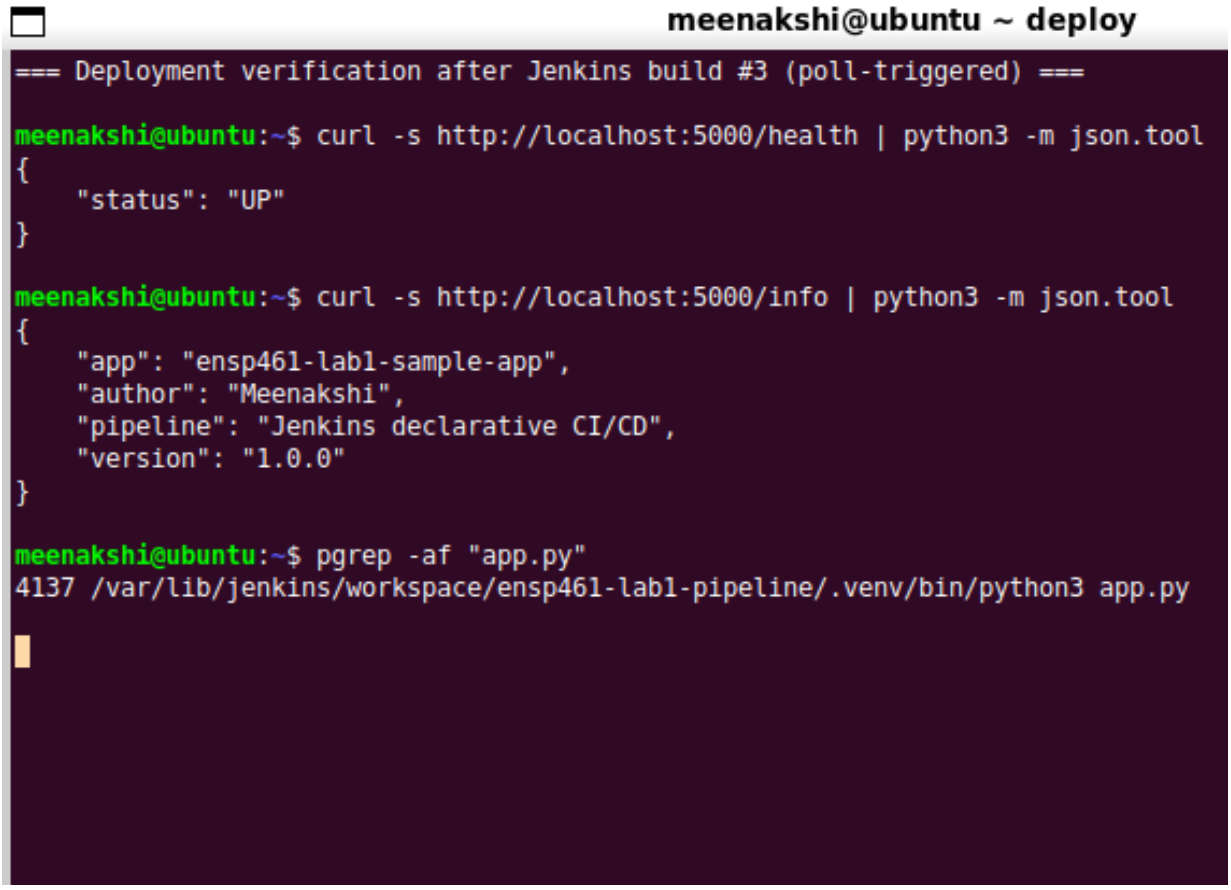


Figure: Deployed Flask application verified on port 5000

8. Pipeline configuration file (Jenkinsfile)

committed at the repository root

```
// ENSP461 Lab 1 - Declarative CI/CD pipeline
// Stages: Build -> Test -> Deploy, triggered from GitHub
pipeline {
    agent any

    environment {
        VENV = "$WORKSPACE/.venv"
        DEPLOY_DIR = "$HOME/enp461-deploy"
    }

    stages {
        stage("Checkout") {
            steps {
                echo "Pulling source code from GitHub repository..."
                checkout scm
            }
        }

        stage("Build") {
            steps {
                echo "Building application: creating virtual environment and installing \
      dependencies..."
                sh """
                    python3 -m venv $VENV
                    $VENV/bin/pip install --quiet -r app/requirements.txt
                """
            }
        }

        stage("Test") {
            steps {
                echo "Running unit tests with pytest..."
```

```
                sh "$VENV/bin/python -m pytest app/tests -v"
            }
        }

        stage("Deploy") {
            steps {
                echo "Deploying application to local deployment directory..."
                sh "./deploy/deploy.sh"
            }
        }
    }

    post {
        success {
            echo "Pipeline SUCCESS: application built, tested and deployed."
        }
        failure {
            echo "Pipeline FAILED - check the stage logs."
        }
        always {
            echo "Cleaning workspace temporary files..."
            cleanWs()
        }
    }
}
```

9. Observations (summary)

- Build #1 failed in the Deploy stage (curl exit code 7): the deployment script invoked the system python3, which does not have Flask installed - diagnosed from the console log (ModuleNotFoundError: No module named 'flask').
- Fix: deploy.sh now reuses the virtual environment created in the Build stage (VENV) and retries the health check; committed as c9df47c, after which build #2 succeeded.
- Jenkins' process-tree killer stopped the deployed app after the build ended; setting JENKINS_NODE_COOKIE=dontKillMe keeps it running (commit eaa01d3).
- Build #3 fired with cause Started by an SCM change - the Poll SCM trigger automates the whole chain from git push to deployment without manual steps.
- Meaningful commit messages keep the history self-documenting; the pipeline is reproducible from the repository alone (Jenkinsfile + deploy script).